



Syed Alyan Ali Hashmi

**ARTIFICIAL INTELLIGENCE (MACHINE LEARNING & DEEP
LEARNING)**

NAVTTC PMYSDP

LECTURE No 09

SIR SUHAIL IMRAN

Method

Procedure

```
def my_procedure(word):  
    print(f"Hello {word} : From my Procedure!")
```

```
my_procedure("aliyan")
```

```
Hello aliyan : From my Procedure!
```

```
y = my_procedure("aliyan") # the pocedure can not be stored. That's why we are getting "None"  
print(y)
```

```
Hello aliyan : From my Procedure!  
None
```

Function

```
def calc(num1, num2, operator):  
    """ num1 : int  
        num2 : int  
        operator : + - * /  
    """  
    if operator == "+":  
        result = num1 + num2  
    elif operator == "-":  
        result = num1 - num2  
    elif operator == "*":  
        result = num1 * num2  
    elif operator == "/":  
        result = num1 / num2  
    else:  
        result = "Invalid Operator"  
  
    return result
```

```
calc(5,10,"+")
```

```
15
```

```
calc(5,10,"-")
```

```
-5
```

```
calc(5,10,"*")
```

```
50
```

```
calc(5,10,"/")
```

```
0.5
```

```
calc(2,3,"+")
```

```
5
```

```
def summation(list):  
    sum = 0  
    for i in list:  
        sum = sum + i  
    return sum
```

```
l = [2,4,5,67,7]  
summation(l)
```

```
85
```

```
summation([1,2,3,4,5])
```

```
15
```

```
def great(list):  
    maximum = list[0]  
    for i in list:  
        if i > maximum:  
            maximum = i  
    return maximum
```

```
great([4,1,9,11,2,4, 100])
```

```
100
```

```
def least(list):  
    minimum = list[0]  
    for i in list:  
        if i < minimum:  
            minimum = i  
    return minimum
```

```
least([4,1,9,11,2,4])
```

```
1
```

```
def temp_conversion(input_temp, input_scale, output_scale):  
    if input_scale in ["c", "C"] and output_scale in ["F", "f"]:  
        result = (input_temp * 9/5) + 32  
    elif (input_scale in ["f", "F"]) and (output_scale in ["C", "c"]):  
        result = (input_temp - 32) * 5/9  
    elif (input_scale in ["C", "c"]) and (output_scale in ["K", "k"]):  
        result = input_temp + 273  
    elif (input_scale in ["K", "k"]) and (output_scale in ["C", "c"]):  
        result = input_temp - 273  
    elif (input_scale in ["F", "f"]) and (output_scale in ["K", "k"]):  
        result = (input_temp - 32) * 5/9 + 273  
    elif (input_scale in ["K", "k"]) and (output_scale in ["F", "f"]):  
        result = (input_temp - 273) * 9/5 + 32  
    else:  
        result = "Invalid Input Unit"  
    return result
```

```
temp_conversion(273, "K", "C")
```

```
0
```

```
temp_conversion(100, "C", "c")
```

```
'Invalid Input Unit'
```

▼ Multiplication Table

```
def table(number=2, start=1, end=10):  
    for i in range(start,end+1):  
        print(f"{number} x {i} = {number * i}")
```

```
table(7)
```

```
7 x 1 = 7  
7 x 2 = 14  
7 x 3 = 21  
7 x 4 = 28  
7 x 5 = 35  
7 x 6 = 42  
7 x 7 = 49  
7 x 8 = 56  
7 x 9 = 63  
7 x 10 = 70
```

```
def pattern(pattern_name="diamond", rows=6):  
    pattern = pattern_name.lower()  
    rows += 1  
    n = rows
```

```

# Pattern 1 - Diamond
if pattern == "diamond":
    for i in range(1, n, 1):
        print((n-i) * " " + i * "#", end="\b")
        print(i * "#")
    for i in range(n, 0, -1):
        print((n-i) * " " + i * "#", end="\b")
        print(i * "#")
# Pattern 2 - Upper Triangle
elif pattern == "upper triangle":
    for i in range(1, n, 1):
        print((n-i) * " " + i * "#", end="\b")
        print(i * "#")
# Pattern 3 - Lower Triangle
elif pattern == "lower triangle":
    for i in range(n, 0, -1):
        print((n-i) * " " + i * "#", end="\b")
        print(i * "#")
# Pattern 4 - Upper Right Triangle
elif pattern == "upper right triangle":
    for i in range(1,n,1):
        print((n-i) * " " + i * "#")
# Pattern 6 - Lower Right Triangle
elif pattern == "lower right triangle":
    for i in range(1,n,1):
        print(i * " " + (n-i) * "#")

else:
    print("Invalid Pattern")

```

```
pattern("upper triangle", 10)
```

```

      #
     ##
    ###
   ####
  #####
 #####
#####
#####
#####
#####
#####
#####
#####
#####

```

Start coding or [generate](#) with AI.